RESQNET
INTELLIGENT EMERGENCY RESPONSE & DISASTER EVACUATION OPTIMIZATION SYSTEM

A PROJECT REPORT

Submitted by

J SRIKAR
UID: 25BCS12640

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

NOVEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “ResQNet — Intelligent Emergency Response & Disaster Evacuation Optimization System” is a bonafide work carried out by J Srikar (UID: 25BCS12640), a student of Bachelor of Engineering in Computer Science and Engineering at Chandigarh University, in partial fulfillment of the requirements for the Analysis and Design of Algorithms (ADSA) subject.

The work is an academic simulation developed and documented for educational purposes. The certificate is provided as a blank institutional form for review; no approval or signature is implied.

Project Supervisor	Head of Department
Signature: _____	Signature: _____
Name / designation: [To be completed by university]	Name / designation: [To be completed by university]
Date: _____	Date: _____

Table 1. Signature and academic designation placeholders

Internal Examiner	External Examiner
Signature: _____	Signature: _____
Name: [To be completed]	Name: [To be completed]

Table 2. Examiner signature placeholders

Viva-voce examination date: _____

TABLE OF CONTENTS

BONAFIDE CERTIFICATE....2
TABLE OF CONTENTS....3
LIST OF FIGURES....5
LIST OF TABLES....6
LIST OF STANDARDS....7
ABSTRACT....8
GRAPHICAL ABSTRACT....9
ABBREVIATIONS....10
SYMBOLS....10
CHAPTER 1 INTRODUCTION....11
1.1 Identification of Client / Need / Relevant Contemporary Issue....11
1.2 Identification of Problem....11
1.3 Identification of Tasks....11
1.4 Timeline....12
1.5 Organization of the Report....12
CHAPTER 2 LITERATURE REVIEW / BACKGROUND STUDY....14
2.1 Timeline of the Reported Problem....14
2.2 Existing Solutions....14
2.3 Bibliometric Analysis....15
2.4 Review Summary....15
2.5 Problem Definition....15
2.6 Goals / Objectives....15
CHAPTER 3 DESIGN FLOW / PROCESS....16
3.1 Evaluation and Selection of Specifications / Features....16
3.2 Design Constraints....16
3.3 Analysis of Features and Finalization Subject to Constraints....17
3.4 Design Flow....17
3.5 Design Selection....18
3.6 Implementation Plan / Methodology....18
Graph and database model....20
Dispatch and care workflow....21
Routing, evacuation and infrastructure methodology....22
Testing and deployment methodology....23
CHAPTER 4 RESULTS ANALYSIS AND VALIDATION....24
4.1 Implementation of Solution....24
4.1.1 Development Environment....24
4.1.2 Graph Representation....24
4.1.3 Database Implementation....24
4.1.4 Algorithm Implementations....24

Dijkstra with binary heap....24
Dijkstra with array minimum selection....24
A* Search....25
BFS and DFS....25
Kruskal minimum spanning forest....25
Edmonds-Karp maximum flow....25
Bellman-Ford....25
4.1.5 Emergency Dispatch Implementation....25
4.1.6 Route Optimization Implementation....25
4.1.7 Hospital Allocation Implementation....26
4.1.8 Dynamic Road Management....26
4.1.9 Evacuation Simulation....26
4.1.10 Algorithm Visualization....26
4.1.11 Performance Analysis....26
4.1.12 Testing and Validation....27
4.1.13 Application Interface Screenshots....27
CHAPTER 5 CONCLUSION AND FUTURE WORK....33
5.1 Conclusion....33
5.2 Future Work....33
REFERENCES....34
APPENDIX A — IMPLEMENTATION AND TEST EVIDENCE....35
APPENDIX B — USER MANUAL....35
B.1 Installation and startup....35
B.2 Dashboard and map....35
B.3 Dispatch and incident status....35
B.4 Routing and algorithm lab....35
B.5 Hospital recommendation....35
B.6 Evacuation....35
B.7 Network, analytics, reset and safety....35

LIST OF FIGURES

Figure 1. Graphical abstract: incident-to-analysis simulation workflow 9
Figure 2. Estimated project development Gantt chart (not historical) 12
Figure 3. Selected milestones in algorithms and emergency coordination 12
Figure 4. Chronological overview of selected algorithm and emergency-management milestones 14
Figure 5. Alternative 1: monolithic application structure 17
Figure 6. Alternative 2: modular Flask architecture 18
Figure 7. Overall implemented ResQNet architecture 18
Figure 8. Use-case view for a simulation operator 19
Figure 9. System request and result workflow 19
Figure 10. Seed graph and directed road representation 19
Figure 11. SQLite entity model (selected fields and relationships) 20
Figure 12. API request lifecycle and validation path 20
Figure 13. Dispatch eligibility, route ranking and assignment flow 21
Figure 14. Hospital recommendation and reservation flow 21
Figure 15. Shortest-path service and route reconstruction 22
Figure 16. Evacuation network construction and max-flow workflow 22
Figure 17. Road closure and subsequent route recalculation 22
Figure 18. Measured benchmark growth sweep (log scale) 27
Figure 19. Command overview and simulated network KPIs 28
Figure 20. Metroville SVG city network 28
Figure 21. Simulated incident dispatch center 29
Figure 22. Route optimization panel 29
Figure 23. Hospital recommendation and allocation panel 30
Figure 24. Capacity-based evacuation planner 30
Figure 25. Algorithm trace and visualization lab 31
Figure 26. Algorithm benchmarking interface 31
Figure 27. Network management interface 32
Figure 28. Simulation analytics 32

LIST OF TABLES

Table 1. Signature and academic designation placeholders 2
Table 2. Examiner signature placeholders 2
Table 3. Standards considered as design guidance 7
Table 4. Abbreviations used in this report 10
Table 5. Mathematical symbols used in this report 10
Table 6. Estimated project work plan 12
Table 7. Qualitative comparison of related approaches 15
Table 8. Requirements and implementation status 16
Table 9. Feature evaluation and scope decisions 17
Table 10. Architecture alternatives comparison 18
Table 11. Deterministic Metroville seed inventory 24
Table 12. Benchmark medians (milliseconds / edge checks) 26
Table 13. Verification summary 27
Table 14. Key source modules and responsibilities 35

LIST OF STANDARDS

The standards below were considered as design and terminology guidance. This project has not undergone a formal conformity assessment and does not claim certification or compliance.

Standard	Publishing agency	About the standard	Page Number
ISO/IEC 25010:2023, Systems and software engineering — SQuaRE — Product quality model (2023 edition)	ISO / IEC	Quality-model vocabulary used to discuss functional suitability, usability, reliability, performance efficiency, maintainability and security. Guidance only; no conformance test was performed.	8
ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering	ISO / IEC / IEEE	Requirements-engineering guidance considered when organizing functional and quality requirements. No formal process compliance is asserted.	8
ISO 22320:2018, Security and resilience — Emergency management — Guidelines for incident management	ISO	Incident-management concepts provide background for the simulated incident workflow. ResQNet is not an operational incident-management system.	8

Table 3. Standards considered as design guidance

ABSTRACT

Emergency response and evacuation planning require decisions across a road network whose routes, facility constraints and movement capacities may change. ResQNet is an academic, offline simulation that models the fictional city of Metroville as a weighted directed graph and demonstrates the application of graph algorithms to routing, ambulance selection, hospital recommendation, road closure analysis and capacity-limited evacuation.

The deterministic seed contains 36 locations and 120 directed roads, alongside simulated ambulances, hospitals, shelters and training incidents. Each directed road stores distance, travel time, evacuation capacity, open/closed state and an infrastructure-cost value. The Flask application separates pure algorithms from service logic, HTTP validation and SQLite persistence. Its browser interface is implemented with HTML, CSS, vanilla JavaScript and SVG.

The implemented algorithms include binary-heap and array-selection Dijkstra, A* with a geographic lower-bound heuristic, BFS, DFS, Kruskal with a disjoint-set structure, Edmonds-Karp maximum flow, and Bellman-Ford in an isolated educational sandbox. Representative measured results include a 20.5-minute Dijkstra route from M01 to M36, a simulated dispatch choice of AMB-07 for incident 1, and an evacuation maximum flow of 480 against a demand of 550 in the captured test scenario. The automated test suite completed with 16 passed tests.

The results demonstrate how algorithmic models and instrumented counters can be connected to interactive workflows. Because the network, traffic and operational data are simulated, and because no real dispatch integration, live traffic or validated safety model exists, ResQNet must not be used for real emergency decisions.

GRAPHICAL ABSTRACT

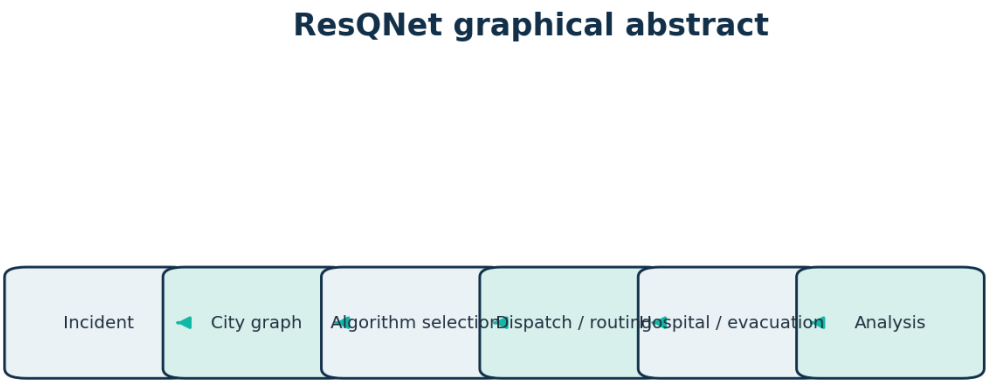


Figure 1. Graphical abstract: incident-to-analysis simulation workflow

The application turns a simulated incident into a graph problem, selects a relevant algorithm, and returns a route, allocation or capacity result with observable statistics.

ABBREVIATIONS

Abbreviation	Meaning
ADSA	Analysis and Design of Algorithms
A*	A-star search
API	Application Programming Interface
BFS	Breadth-First Search
CRUD	Create, Read, Update, Delete
DFS	Depth-First Search
DSU	Disjoint-Set Union
ETA	Estimated Time of Arrival (simulated)
HTTP	Hypertext Transfer Protocol
MST	Minimum Spanning Tree
REST	Representational State Transfer
SVG	Scalable Vector Graphics
UI	User Interface
UX	User Experience
V	Vertex set
E	Edge set

Table 4. Abbreviations used in this report

SYMBOLS

Symbol	Definition
G = (V, E)	A graph with vertex set V and directed edge set E
u, v	Vertices / locations in a graph
w(u,v)	Non-negative route cost on directed edge (u,v), e.g. time or distance
d(v)	Current tentative shortest-path distance from the source to v
g(n)	Known path cost from the source to node n in A*
h(n)	Admissible lower-bound estimate from n to the target in A*
f(n) = g(n) + h(n)	A* priority value
C(u,v)	Capacity of directed edge (u,v), expressed as simulated people per configured period
F	Maximum feasible source-to-sink flow
O(·)	Asymptotic upper-bound notation for algorithmic complexity
V, E	Number of vertices and edges, respectively, in complexity expressions

Table 5. Mathematical symbols used in this report

CHAPTER 1
INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

ResQNet has no external client or commissioning emergency agency. It is an academic simulation for the ADSA subject. The broader need it examines is the coordination challenge of moving response vehicles, patients and evacuees through a constrained, potentially disrupted transport network.

A response plan can be affected by road accessibility, facility capacity, specialty requirements, directional closures and the distinction between travel cost and movement throughput. The World Health Organization's Global Status Report on Road Safety 2023 estimates approximately 1.19 million road-traffic deaths annually worldwide [1]. This statistic establishes the public-health significance of road safety; it is not an estimate of ambulance delay or a direct evaluation of ResQNet.

For Indian context, the National Disaster Management Plan (2019) is a national planning reference for disaster preparedness and response coordination [13]. ResQNet does not implement that plan or represent any Indian agency; the reference only situates the academic topic within the country's documented disaster-management planning framework.

Graph models offer a disciplined way to express locations, one-way connectivity, route weights and capacities. Shortest-path algorithms can compare candidate routes under a stated metric; matching and ranking rules can help demonstrate dispatch and facility selection; max-flow algorithms can reason about throughput under explicit capacity assumptions. These are decision-support models, not substitutes for trained emergency managers, live situational awareness or validated operational protocols.

The project uses fictional Metroville data and makes no claim that its topology, capacity assumptions or results correspond to Chandigarh, India, or any real emergency service.

1.2 Identification of Problem

The problem addressed is the difficulty of coordinating simulated emergency movements across a city network when routes and destinations have different costs and constraints. The problem includes identifying reachable places, comparing travel options, selecting available response resources, considering medical eligibility and estimating how much evacuation demand can traverse constrained roads and shelters.

A road closure may disconnect a location or increase route cost. A hospital may be geographically close but unsuitable because it lacks the required specialty, is not accepting patients or has no available bed. Evacuation throughput is constrained by road and shelter capacities and may not meet the total demand. These interacting constraints make manual comparison difficult and require clear, reproducible representations.

1.3 Identification of Tasks

Development activities: investigate the problem and related literature; define requirements; model the directed graph and domain entities; select algorithms; implement independent algorithm modules; design the SQLite schema and deterministic seed; build Flask services and API routes; develop the browser interface and SVG visualizations; integrate and document deployment configuration.

Verification activities: write unit tests for graph algorithms and edge cases; use Flask's test client for API behavior; exercise representative dispatch, route, hospital, road-closure and evacuation workflows; run the complete pytest suite; inspect the local application and screenshots; report observed limitations separately from completed features.

Work package	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Research and requirements	•	•				
Graph / schema / seed design		•	•			

Work package	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Algorithms and services			•	•		
API and user interface				•	•	
Integration and validation					•	•
Report and final review						•

Table 6. Estimated project work plan

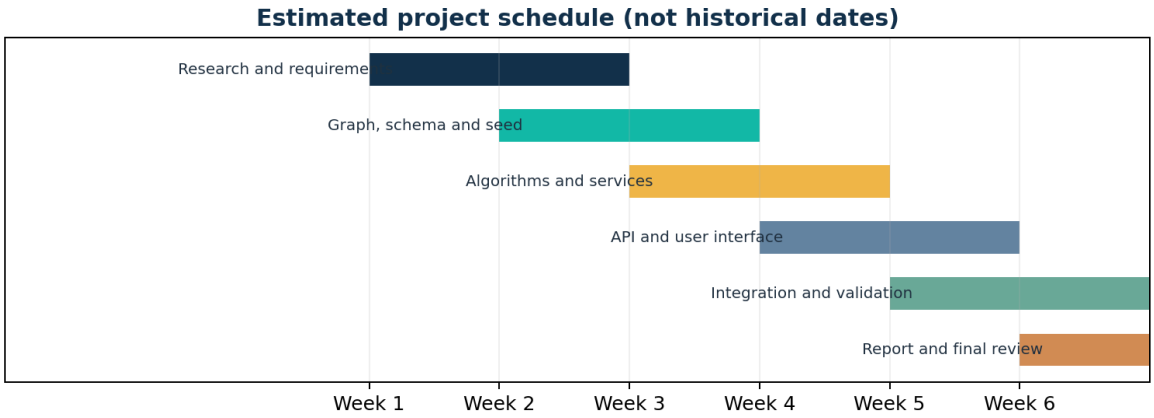


Figure 2. Estimated project development Gantt chart (not historical)

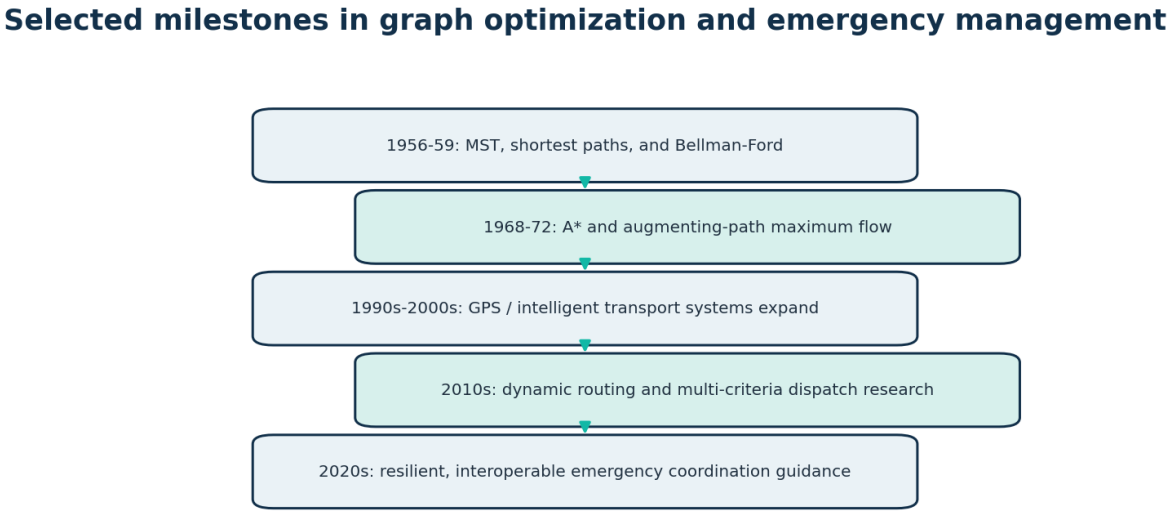


Figure 3. Selected milestones in algorithms and emergency coordination

The repository history does not provide a reliable multi-week development chronology. Therefore, the schedule above is an estimated work plan, not a reconstruction of actual completion dates.

1.4 Timeline

Algorithmic milestones relevant to the project are presented in Figure 2. The report's development schedule is intentionally labeled estimated because the available Git history contains only a single dated repository commit and does not establish a detailed task-by-task chronology.

The chosen milestones are historical context rather than a quantitative bibliometric analysis. Sources are listed in References [2]-[8].

1.5 Organization of the Report

Chapter 1 introduces the need, problem, tasks and schedule. Chapter 2 reviews foundational work, existing approaches and the technical problem definition. Chapter 3 documents requirements, constraints, alternative designs and the selected implementation process. Chapter 4 presents actual implementation details, algorithm

examples, interface captures, measurements and test outcomes. Chapter 5 summarizes achieved work and proposes future enhancements. References, an implementation appendix and an operator-oriented user manual follow.

CHAPTER 2
LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Graph optimization has developed through several foundational strands. Kruskal's 1956 minimum-spanning-tree method formalized a greedy edge-selection approach [2]. Bellman (1958) and Ford (1956) independently developed dynamic-programming / relaxation foundations for shortest paths with negative edge costs [3, 4]. Dijkstra's 1959 shortest-path algorithm established a label-setting method for non-negative weights [5].

Hart, Nilsson and Raphael introduced A* as a best-first search guided by accumulated cost and heuristic estimate in 1968 [6]. Edmonds and Karp proved a polynomial-time augmenting-path maximum-flow algorithm in 1972 [7]. Later work on intelligent transportation systems, vehicle navigation, dynamic travel costs and emergency logistics extended these graph foundations to changing, multi-criteria settings.

Emergency management guidance increasingly emphasizes coordination, information exchange and incident management. ISO 22320:2018 provides incident-management guidelines [11]. This report uses it as a vocabulary reference, not as evidence of operational compliance.

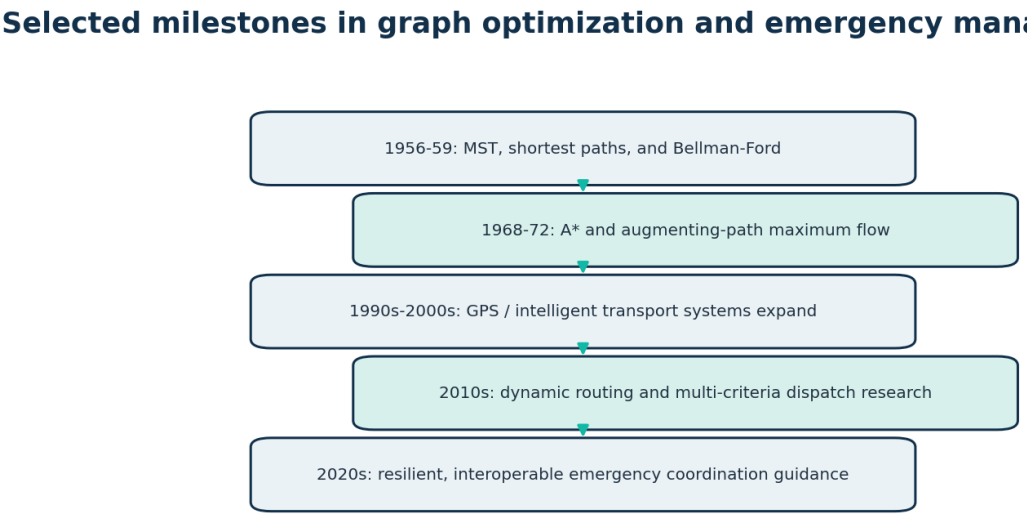


Figure 4. Chronological overview of selected algorithm and emergency-management milestones

2.2 Existing Solutions

Static shortest-path algorithms optimize a fixed graph and non-negative weights; their results are deterministic but only as current as the supplied data. ResQNet routes by simulated time or distance.

Commercial navigation combines maps, positioning and traffic feeds to adapt routes, but depends on external coverage and services. ResQNet uses no live GPS, traffic or paid routing API.

Dispatch methods may jointly consider availability, travel time, coverage and workload. ResQNet instead filters its simulated fleet by capability, computes each eligible unit's ETA and selects the lowest, with ID tie-breaking.

Hospital allocation can use multi-constraint ranking or global optimization. ResQNet filters one incident's options by acceptance, beds, specialty and reachability, then ranks by ETA; its severity-ordered queue remains greedy.

Network-flow models represent evacuation capacity between origins and destinations. ResQNet applies Edmonds-Karp to directed road and shelter capacities, without time-expanded flow, congestion or behavioral modelling.

2.3 Bibliometric Analysis

A citation-index dataset and publication-count extraction were not assembled. Accordingly, this section is a qualitative comparative literature analysis, not a quantitative bibliometric study. The following comparison summarizes canonical algorithm families and system approaches without inventing citation counts or claiming comparative field-wide effectiveness.

Approach	Core technique	Strength	Limitation / ResQNet relevance
Static shortest path	Dijkstra / A* on fixed non-negative weights	Clear optimality under stated graph assumptions	Needs current graph and cost; does not itself assign vehicles
Negative-weight shortest paths	Bellman-Ford edge relaxation	Supports negative weights and detects reachable negative cycles	Slower; negative travel times are not physically meaningful for the road workflow
Dispatch optimization	Eligibility filtering + route ETA ranking	Operationally interpretable candidate ranking	Single-incident greedy rule; no fleet-wide optimization
Facility matching	Capacity / specialty / ETA constraints	Rejects unsuitable destinations explicitly	No multi-patient global assignment or clinical triage model
Evacuation network flow	Augmenting paths / max flow	Identifies throughput bottlenecks under capacities	Static capacity abstraction; no time-varying demand or behavioral model
Live navigation	Traffic feeds + route algorithms	Can adapt to observed conditions	External data/services and operational validation required; outside project scope

Table 7. Qualitative comparison of related approaches

2.4 Review Summary

The literature provides robust algorithmic building blocks, but algorithmic optimality is conditional on model quality. A shortest route is optimal only for the chosen edge weights; maximum flow is optimal only for the supplied capacities and network. Real emergency coordination additionally requires reliable current data, clinical and operational protocols, resilient communications and human oversight.

ResQNet contributes an educational environment in which different graph problems can be separated, visualized and tested on a reproducible city network. The comparison of Dijkstra variants, A*, traversals, spanning forests, flow and Bellman-Ford makes their assumptions and measured counters inspectable.

2.5 Problem Definition

Let G = (V, E) be Metroville's directed graph. A vertex v ∈ V represents a location, intersection or facility. A directed edge (u,v) ∈ E represents one road direction and stores non-negative distance d(u,v), travel time t(u,v), evacuation capacity c(u,v), infrastructure cost k(u,v), and an open/closed flag. Reciprocal streets are represented by two independent directed edges; closing one direction does not implicitly close the other.

Routing seeks a path P from source s to target t minimizing either Σt(u,v) or Σd(u,v), over currently open edges. Dispatch selects an available, capability-compatible ambulance with a reachable minimum-time path to an incident. Hospital recommendation filters accepting facilities with a free bed, matching specialty and reachable path, then ranks by ETA. Evacuation augments G with super-source S and sink T; zone demand arcs enter the network and shelter-capacity arcs leave it. The objective is to maximize feasible flow F from S to T.

Scope excludes live traffic, GPS positioning, actual emergency dispatch, real patient data, verified clinical triage, uncertainty, time-expanded flow, and proof that model capacities correspond to real throughput. Infrastructure planning uses Kruskal on an undirected projection of open roads; it is a connectivity planning illustration, not a validated construction plan.

2.6 Goals / Objectives

Implemented objectives include a reproducible SQLite city graph; route calculation and comparison using heap Dijkstra, array Dijkstra and A*; eligible ambulance dispatch; hospital ranking and bed reservation; road-closure rerouting; reachability and Kruskal analysis; Edmonds-Karp evacuation flow with minimum cut; trace visualization; and seeded performance benchmarks. The 16-test suite validates core algorithms and APIs. Live emergency response and real-world safety certification are out of scope.

CHAPTER 3
DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

Requirements were evaluated for academic algorithm coverage, reproducibility, safety boundaries, implementability without external services and observable user workflows. Status refers to the inspected implementation and browser/test evidence.

ID	Requirement	Priority	Status / evidence
FR-01	Seed deterministic city locations, directed roads, facilities and scenarios	High	Implemented; 36 locations / 120 roads
FR-02	Compute route by time or distance with Dijkstra heap / array or A*	High	Implemented and tested
FR-03	Dispatch available capable ambulance by ETA	High	Implemented; transactional persistence
FR-04	Recommend hospital by eligibility and route ETA	High	Implemented; bed reservation workflow
FR-05	Close / reopen road and use updated graph snapshots	High	Backend and route workflow implemented
FR-06	Evacuation max flow with capacities and min cut	High	Implemented and tested
FR-07	Traversal / connectivity / infrastructure analysis	Medium	Algorithms and API implemented; form submission issue observed
FR-08	Trace algorithm and show counters	Medium	Implemented for supported algorithms
FR-09	Benchmark repeated runs on seeded graphs	Medium	Implemented; measured growth sweep
NFR-01	Run locally without external APIs or keys	High	Implemented
NFR-02	Use persistent SQLite with consistency checks	High	Implemented
NFR-03	Be safe to interpret as real emergency guidance	High	Simulation disclaimer present; never operationally certified
NFR-04	Usable interactive visual interface	Medium	Implemented; UI form defect in connectivity source selector

Table 8. Requirements and implementation status

3.2 Design Constraints

Technical: a single-process Flask application and SQLite database suit a compact academic demonstration but are not designed for high concurrency, multi-region failover or large-scale real-time routing. A directed edge snapshot is in-memory for each service calculation. Benchmark times are machine/runtime-specific.

Economic and environmental: Python, Flask, SQLite and browser-native technologies minimize license and infrastructure costs. The application has no external mapping or AI API dependency. No claim is made about measured energy consumption or environmental benefit.

Health, safety and ethical: this is a simulation, not a certified emergency-dispatch or navigation system. Results must not guide a real incident. The fictional network and capacities are not safety-calibrated. No actual patient-identifying information is needed or included.

Social and operational: simulated resources do not represent actual service availability, local language requirements, accessibility needs or institutional command structures. Human oversight and local emergency protocols would be indispensable in any real deployment.

Security and privacy: the project uses local simulation data and avoids live personal records. It is not a security-certified system; deployment would need authentication, authorization, transport security, logging policy, threat analysis and data retention controls.

Deployment and persistence: Render configuration specifies a persistent-disk mount for SQLite. A local demo is persistent on disk, but deploy platform behavior and operational backup/recovery require separate validation. The app has no external service-level guarantee.

3.3 Analysis of Features and Finalization Subject to Constraints

Features were retained when they demonstrated a distinct algorithmic problem with measurable inputs and outputs. A* and Dijkstra use travel-time/distance weights; maximum flow uses capacity and is not conflated with route cost. Bellman-Ford is kept in an educational sandbox rather than applying negative weights to physical travel. Dynamic road changes are modeled as open/closed edge state, not as live traffic.

Live maps, traffic, GPS, real dispatch integrations, clinical decision rules and AI prediction were excluded because they would require external data, safety validation and privacy/operational governance beyond this academic scope. The connectivity form's missing `source` field name was discovered during manual UI verification and is disclosed as an implementation defect.

Feature	Decision	Reason / status
Deterministic grid dataset	Include	Reproducible algorithm demonstrations
Actual dispatch and bed reservation	Include	Shows business rules plus transactional state
Road closure rerouting	Include	Tests graph snapshot changes
Maximum-flow evacuation	Include	Distinct capacity optimization problem
Live maps and traffic	Exclude	No external APIs; no real-world feeds
Global hospital queue optimization	Defer	Current algorithm is greedy, not globally optimal
Connectivity user form	Partial	Backend works; UI source selector missing submitted name
Real emergency deployment	Exclude	Not certified or validated for operational use

Table 9. Feature evaluation and scope decisions

3.4 Design Flow

Two alternatives were considered. Alternative 1 places UI handling, rules, graph algorithms and persistence in a monolithic application unit. It reduces initial scaffolding but makes algorithm isolation, testing and change-impact control more difficult. Alternative 2 separates the browser, API, services, pure algorithms and SQLite repository. The latter introduces modest module boundaries but matches the project's need to demonstrate algorithms independently and connect each algorithm to a business workflow.

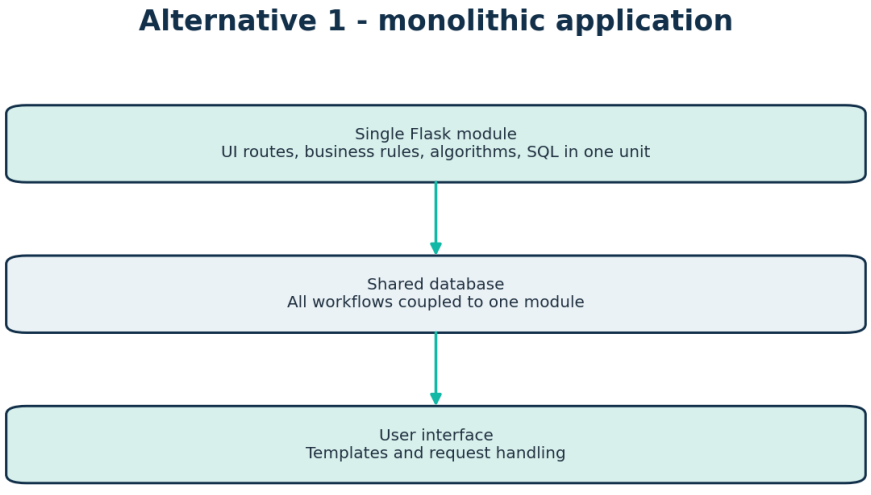


Figure 5. Alternative 1: monolithic application structure

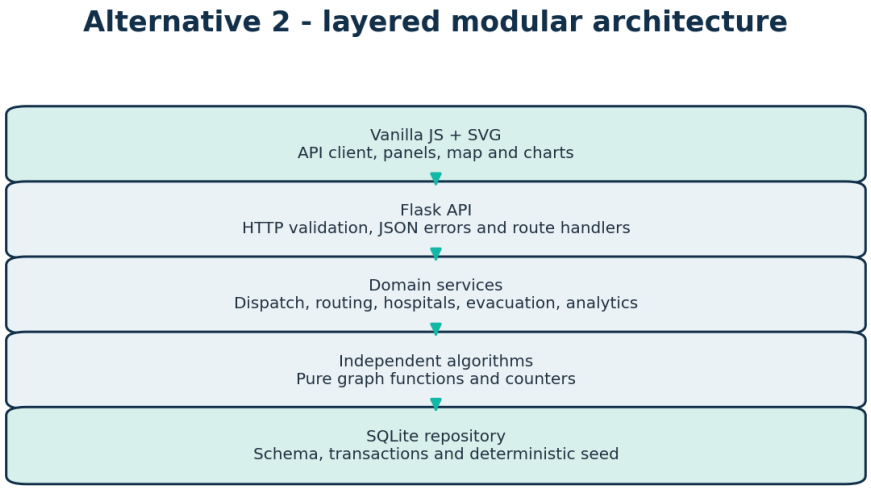


Figure 6. Alternative 2: modular Flask architecture

3.5 Design Selection

The implemented application follows the modular alternative: algorithm functions accept in-memory inputs; services load graph snapshots and coordinate domain workflows; database modules own schema and repository operations; Flask routes validate and serialize; browser code renders returned data. This choice supports unit testing of algorithms, explicit workflow rules and traceable API behavior.

Criterion	Monolithic	Modular implementation
Modularity	Low; concerns tightly coupled	High; algorithms, services, API and persistence separated
Testing	More setup for isolated algorithms	Pure algorithms independently testable
Maintainability	Changes have broad impact	Boundaries localize changes
Initial effort	Lower	Moderate
Scale-out	Limited	Still a compact monolith at deployment; boundaries ease later extraction
ADSA suitability	Algorithm internals harder to isolate	Strong; algorithms are directly inspectable and measurable

Table 10. Architecture alternatives comparison

3.6 Implementation Plan / Methodology

Overall architecture and API workflow are shown in Figures 6 and 7. A service reads a consistent SQLite snapshot, constructs an adjacency mapping of open edges, calls an independent algorithm and returns JSON. State-changing workflows use transaction boundaries and conditional updates where necessary.

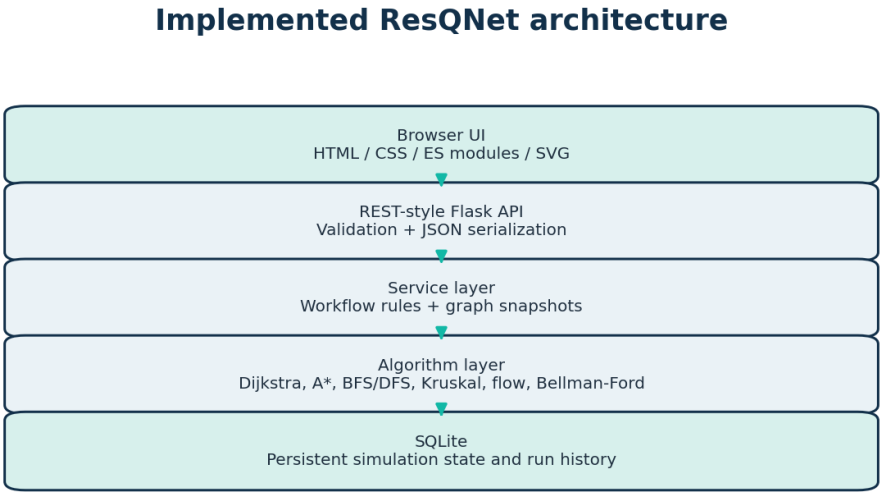


Figure 7. Overall implemented ResQNet architecture

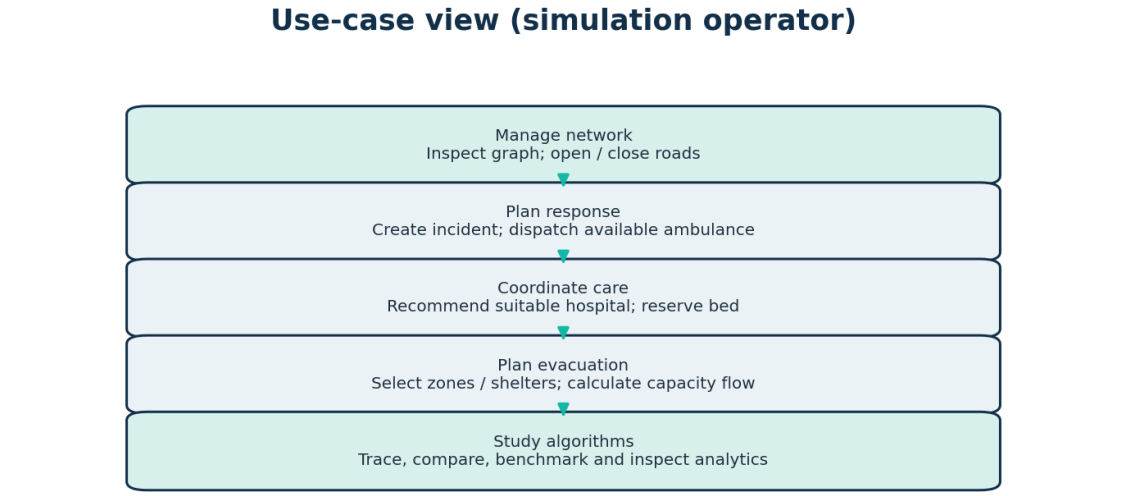


Figure 8. Use-case view for a simulation operator

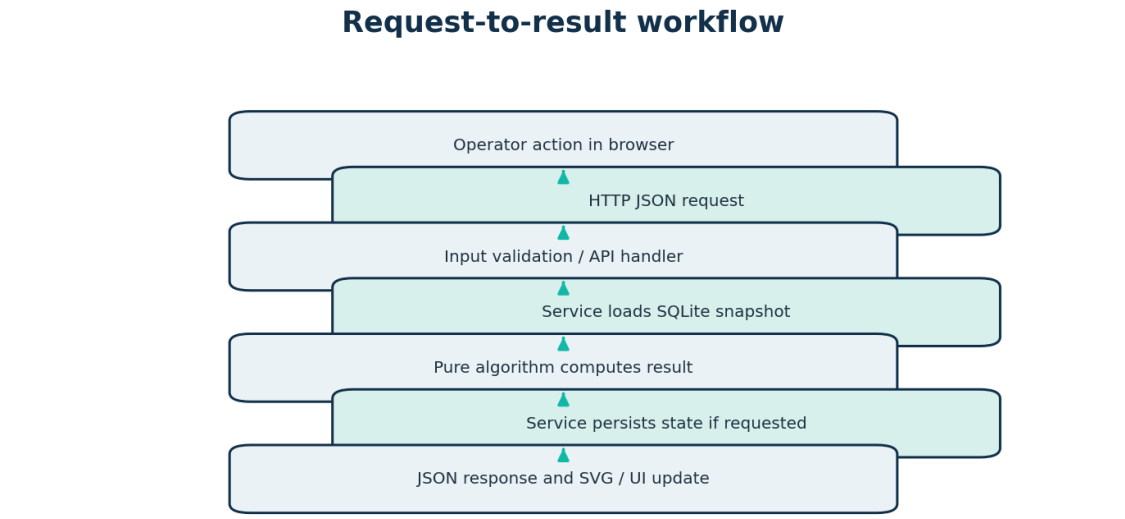


Figure 9. System request and result workflow

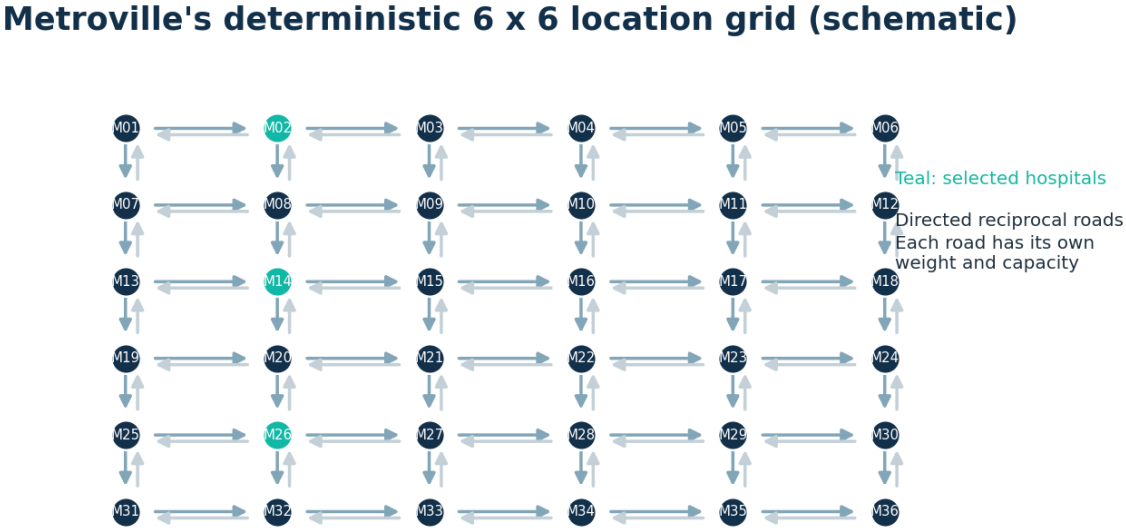


Figure 10. Seed graph and directed road representation

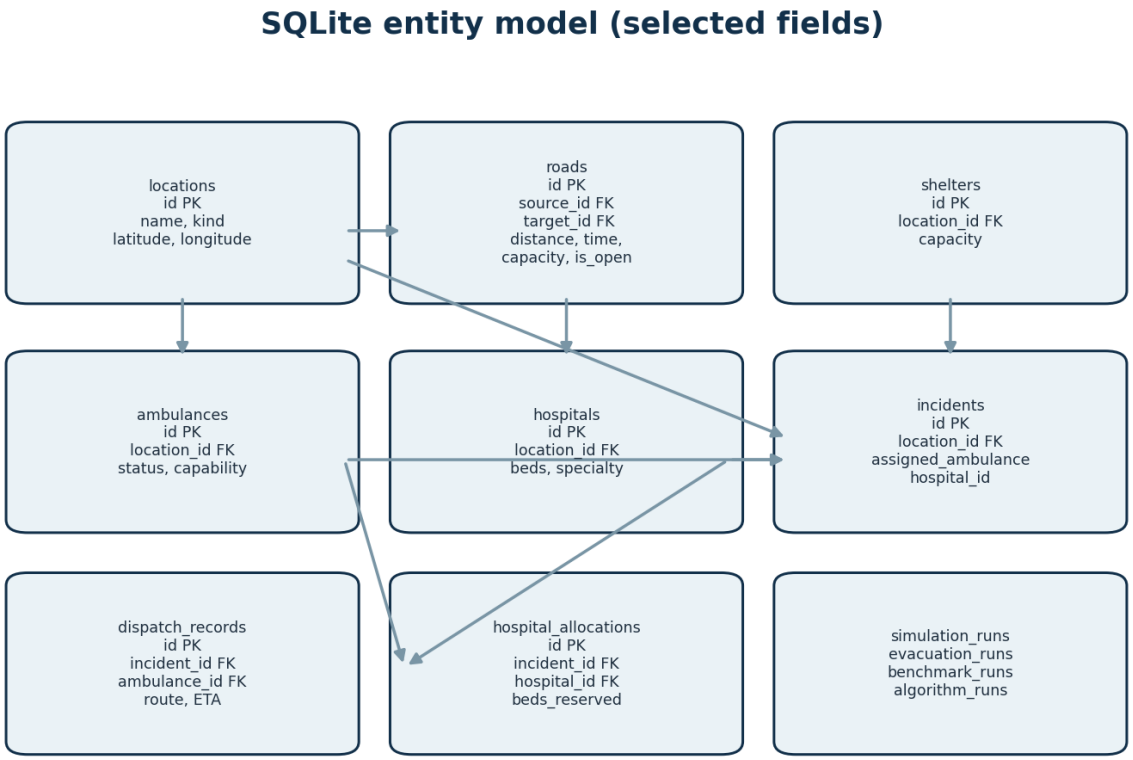


Figure 11. SQLite entity model (selected fields and relationships)

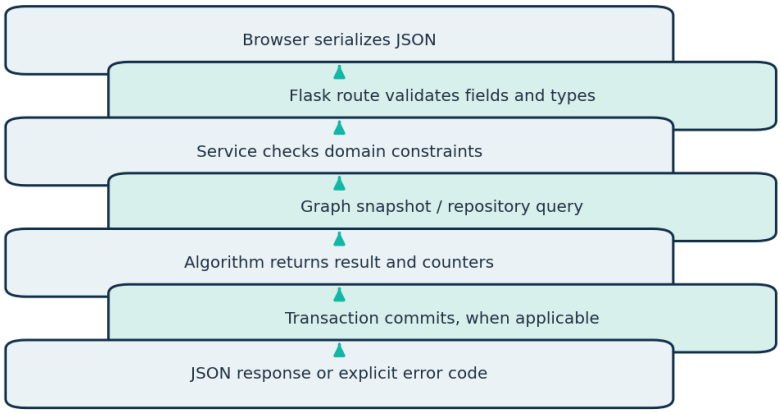


Figure 12. API request lifecycle and validation path

Graph and database model

The `locations` table stores stable IDs, location types and coordinates. `roads` references source and target locations and enforces positive distance/travel time, non-negative capacity and unique direction pairs. Operational tables include ambulances, hospitals, shelters and incidents; dispatch records and hospital allocations capture workflow outcomes. Evacuation, benchmark and algorithm runs store JSON result payloads. Foreign keys, indexes and transaction helpers support consistency. Bidirectional streets are two rows.

Dispatch and care workflow

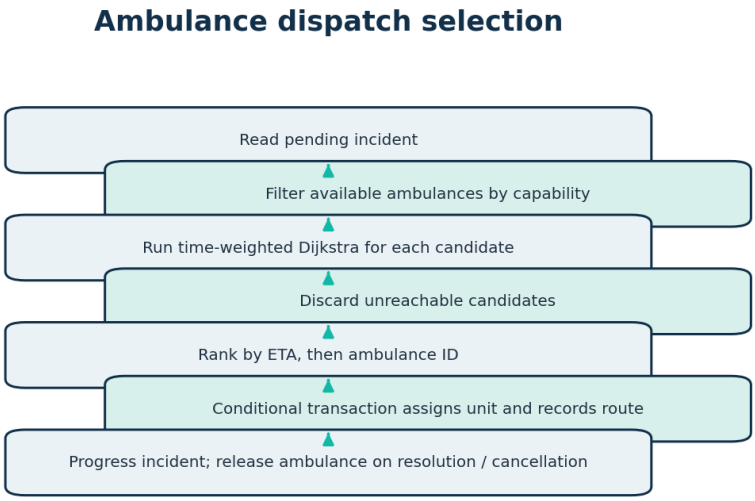


Figure 13. Dispatch eligibility, route ranking and assignment flow

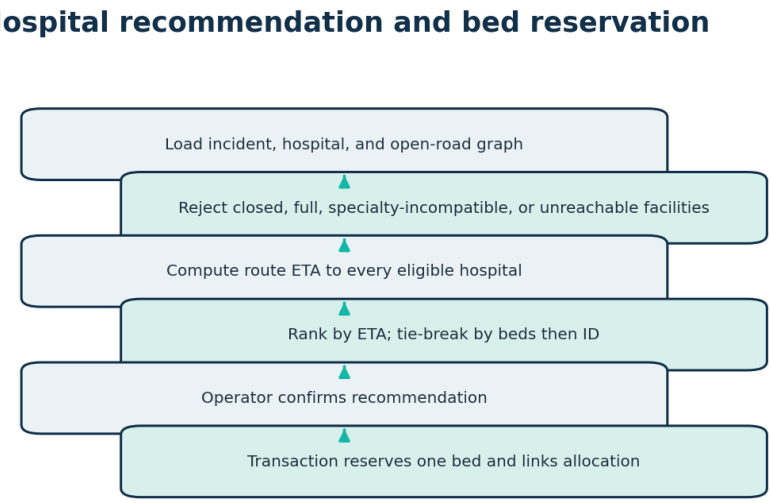


Figure 14. Hospital recommendation and reservation flow

Dispatch filters to available ambulances with no assigned incident, checks requested capabilities, runs time-weighted Dijkstra from each unit to the incident, removes unreachable candidates and orders by ETA then ID. Confirmation conditionally updates both incident and unit, records the route and commits. Hospital recommendations filter accepting status, bed availability, specialty and reachability, then rank by ETA, available beds descending and ID. Reservation checks capacity and incident allocation within a transaction. The batch queue orders incidents by severity and time but uses a greedy per-incident choice.

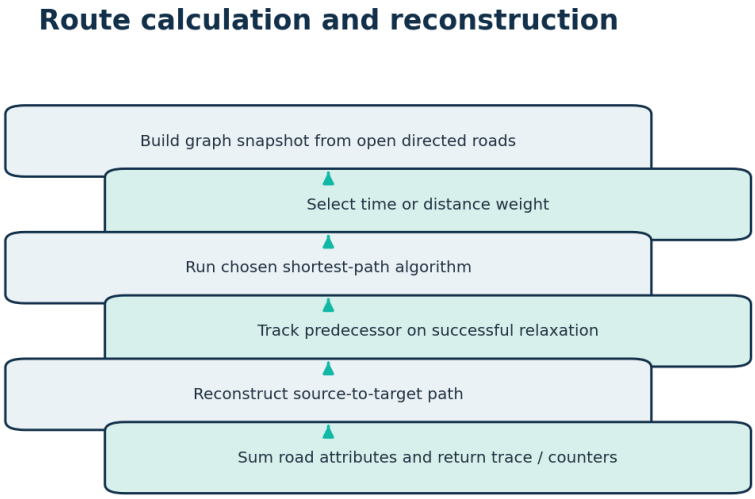


Figure 15. Shortest-path service and route reconstruction

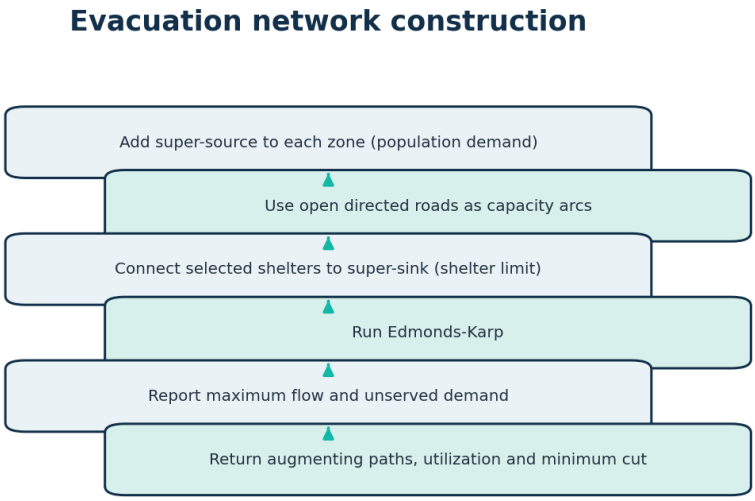


Figure 16. Evacuation network construction and max-flow workflow

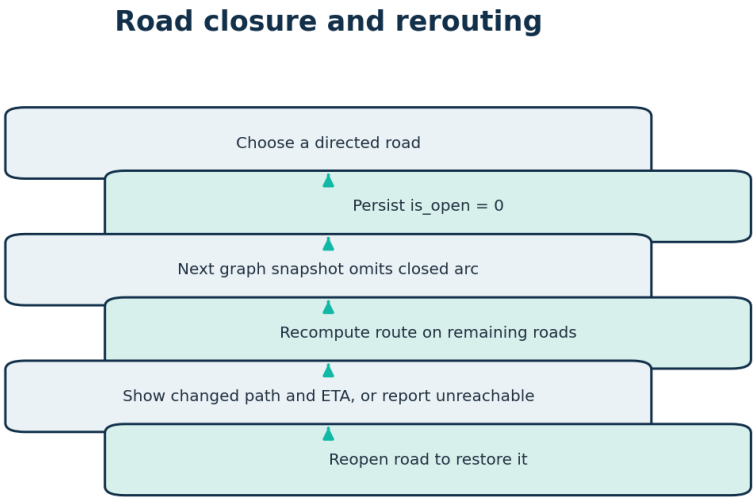


Figure 17. Road closure and subsequent route recalculation

The evacuation service adds a super-source edge to each zone with capacity equal to population demand, uses each currently open directed road's evacuation capacity, and adds a shelter-to-super-sink edge bounded by shelter capacity. Edmonds-Karp returns maximum flow, augmenting paths, used capacities and residual-reachability minimum cut. The capacities are simulated people per configured period, not measured road throughput.

Testing and deployment methodology

Unit tests cover algorithm outputs, invalid inputs, disconnected graphs, negative weights/cycles, residual reverse edges, min-cut equality and boundary cases. Flask test-client tests exercise API validation and service integration. Browser workflows were manually captured. The WSGI entry point uses Flask and the deployment manifest configures Gunicorn and persistent storage; local testing does not establish production reliability.

CHAPTER 4
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

This chapter reports results from the inspected source, local test suite, test-client workflows, and authentic captures of the running interface. Execution evidence is machine-specific, and all incident/transport data are simulated.

4.1.1 Development Environment

The verified local environment used Python 3.14.3, Flask 3.1.3, pytest 8.4.2, SQLite, Windows, and Microsoft Word 16 for document rendering. The source targets Python 3.10+ and uses Flask, SQLite, Gunicorn, pytest, HTML, CSS, vanilla JavaScript and SVG. No external API key or frontend build system is required.

4.1.2 Graph Representation

The seed generator creates a 6-by-6 grid of M01–M36. Each adjacent horizontal or vertical pair becomes two separate directed roads, producing 120 edge rows. Coordinates are deterministic. For the captured route query M01→M36, heap Dijkstra returned M01 → M02 → M08 → M09 → M15 → M21 → M22 → M28 → M29 → M35 → M36, with total 20.5 minutes and 14.2 km. The route has 10 segments.

Entity	Count	Representation / note
Locations	36	Stable IDs M01-M36; 6 x 6 grid
Directed roads	120	Two directions stored independently
Ambulances	8	Availability and capabilities
Hospitals	6	Beds, specialties, acceptance status
Shelters	6	Configured capacity
Training incidents	2	Fictional seed scenarios

Table 11. Deterministic Metroville seed inventory

4.1.3 Database Implementation

SQLite schema uses locations, roads, ambulances, hospitals, shelters, incidents, dispatch_records, hospital_allocations, evacuation_runs, benchmark_runs, algorithm_runs and simulation_events. Constraints include foreign keys, enumerated unit states, non-negative capacities, valid occupied-bed bounds and unique directed road pairs. The application factory initializes the schema and deterministic seed when empty.

4.1.4 Algorithm Implementations

Dijkstra with binary heap

Input is an adjacency mapping from vertex IDs to Edge values containing target, time, distance and capacity. The algorithm initializes distance to infinity, predecessor to null and a min-heap with the source. It discards stale queue entries, settles the minimum-cost node, relaxes outgoing edges and reconstructs the path through predecessors. Non-negative selected weights are required. Complexity is O((V + E) log V) with a binary heap and adjacency lists; auxiliary state is O(V + E). It is used by route, dispatch and hospital services.

Example: for M01→M36 using time weights, the path and cost are given in §4.1.2. In one captured run, 118 edge checks, 35 successful relaxations, 36 settled nodes and 71 heap operations were reported; measured execution was 0.0653 ms. This timing is an observation, not a guarantee.

Dijkstra with array minimum selection

The array variant maintains a set of unvisited vertices and selects the minimum tentative distance by scanning it each iteration. Relaxation and predecessor reconstruction match the heap variant. Its time complexity is $O(V^2 + E)$, commonly $O(V^2)$ for adjacency-list use with linear minimum selection, with O(V + E) storage. It is useful for

independent comparison and can be competitive for dense/small graphs. The Metroville comparison returned the same cost 20.5 as heap Dijkstra.

A* Search

A* orders its heap by f(n)=g(n)+h(n). The implementation computes great-circle distance using Haversine coordinates. For distance mode, its heuristic uses the minimum observed road-distance / straight-line ratio; for time mode, it uses straight-line distance divided by maximum observed geometric road speed. This scales the geographic lower bound to remain admissible for the edge attributes. If coordinates or non-negative weights are absent, it rejects the input. Worst-case search can be exponential in general; performance depends on graph and heuristic, with O(V+E) memory in this implementation. It is validated against Dijkstra on fixtures.

BFS and DFS

BFS uses a FIFO deque and DFS an explicit LIFO stack to enumerate vertices reachable along directed outgoing edges. Both return visitation order and sorted unreachable locations in O(V+E) time and O(V) auxiliary memory. `connected_components` separately constructs an undirected projection and traverses it, so it reports weak-style connectivity for a directed road dataset rather than strongly connected components. From M01, BFS reached all 36 seeded locations.

Kruskal minimum spanning forest

The implementation normalizes reciprocal road directions into unordered endpoint pairs, retains the lowest-weight direction for each pair, sorts candidates, and uses a disjoint-set structure with path halving and union by size. It returns a forest, excluded edges, total selected weight and component sets. Complexity is O(E log E) for sorting plus near-constant amortized DSU operations; storage is O(V+E). Open seeded roads produced 35 edges, one component and total distance 47.4 km in the evidence run. It is an undirected connectivity abstraction, not a route or construction recommendation.

Edmonds-Karp maximum flow

The flow algorithm accepts a node list, directed capacity edges, source and sink. It aggregates capacities into a residual matrix and maintains adjacency including reverse residual edges. BFS searches a shortest-in-edges augmenting path; the bottleneck is subtracted from forward residual capacity and added to reverse capacity. When no path remains, residual reachability partitions a minimum cut. Time complexity is $O(VE^2)$; this matrix implementation uses $O(V^2+E)$ memory. The result includes augmentations, edge checks, used capacities, maximum flow and minimum cut.

The canonical unit-test network returned a maximum flow of 23 and a minimum-cut capacity of 23. In the actual Metroville test-client scenario, demand was 550, maximum flow 480, and unserved demand 70. Four augmentations had bottlenecks 165, 135, 125 and 55; the reported minimum cut consisted of 220- and 260-capacity shelter arcs, totaling 480. The service explicitly labels this as simulated throughput.

Bellman-Ford

Bellman-Ford is implemented in a separate educational graph sandbox. It repeatedly relaxes each edge for up to V−1 rounds, stops early if unchanged, then checks for a source-reachable negative cycle and marks downstream affected distances as negative infinity. It runs in O(VE) time and O(V+E) storage. Tests validate a negative-edge shortest path and a reachable negative cycle. It is not used for physical road travel, because negative travel time/distance is invalid.

4.1.5 Emergency Dispatch Implementation

The captured dispatch for incident 1 selected AMB-07 with ETA 10.4 minutes and 7.2 km over route M33 → M27 → M21 → M22 → M16 → M17. Eligibility and routing are recomputed from currently available units and open roads. The captured interface workflow persisted the dispatch, leaving one ambulance assigned in that test database. Completion or cancellation releases the assigned unit; transitions are explicitly checked.

4.1.6 Route Optimization Implementation

For M01→M36, Dijkstra heap chose M01 → M02 → M08 → M09 → M15 → M21 → M22 → M28 → M29 → M35 → M36. Cost was 20.5 minutes with 14.2 km total distance. The heap, array and A* comparison returned

the same 20.5-minute cost in the recorded comparison. The algorithms use a consistent graph snapshot; route segments are reconstructed from predecessor links and their road attributes are summed.

4.1.7 Hospital Allocation Implementation

For the captured trauma incident at M17, H-03 (Metroville Medical Center 3 at M14) was recommended at 6.7 minutes with 56 beds available. H-02, H-04 and H-06 were rejected for missing trauma specialty. H-05 and H-01 remained eligible but had longer ETAs. The browser workflow reserved one bed at H-03. Recommendation is not clinical triage, and queue assignment is greedy.

4.1.8 Dynamic Road Management

Closing directed road 1 changed the M01→M06 route from M01-M02-M03-M04-M05-M06 (11.5 minutes) to M01-M07-M08-M09-M10-M11-M12-M06 (15.9 minutes). Routing snapshots omit closed roads while the network view can show them blocked. The reciprocal direction remains independent. A UI defect was found during manual inspection: the Network Manager connectivity form's source selector lacks the submitted field name, causing its connectivity request to fail with HTTP 400 ('Missing required field(s): source'). The underlying connectivity service and API test pass when valid input is sent directly.

4.1.9 Evacuation Simulation

In the recorded test-client configuration, zones at M03 and M17 supplied total demand 550 and selected shelter capacities formed the sink arcs. The maximum flow was 480, leaving 70 unserved under the configured capacities. The min-cut capacity also equaled 480, validating max-flow/min-cut equality for this execution. Reported selected augmenting paths and cut are model results, not evacuation guidance.

4.1.10 Algorithm Visualization

The frontend receives trace events produced by backend algorithms and animates visited nodes and distance updates on the SVG graph. For a Dijkstra trace the evidence contained 71 events; the first events visited M01 and updated M02 and M07. The visual layer does not independently compute routes. Screenshots in §4.1.12 are authentic captures of the running interface.

4.1.11 Performance Analysis

A seeded benchmark growth sweep used graph sizes 25, 50, 100, 250 and 500, average degree 4, random seed 42 and three repetitions. Each algorithm was warmed up; timings are medians, tracing disabled. The generated undirected pairs are represented as directed edges. The following are captured local measurements in milliseconds and edge checks.

Vertices	Directed edges	Dijkstra heap	Dijkstra array	A*	Bellman-Ford
25	100	0.0360 / 78	0.0627 / 78	0.1559 / 78	0.1481 / 500
50	200	0.0749 / 193	0.1797 / 193	0.3216 / 193	0.3950 / 1400
100	400	0.3161 / 366	0.5466 / 366	0.6144 / 362	0.9457 / 2800
250	1000	0.2882 / 626	2.1802 / 626	1.5282 / 605	3.4742 / 8000
500	2000	0.7353 / 1819	13.8022 / 1819	3.6004 / 1806	11.8295 / 20000

Table 12. Benchmark medians (milliseconds / edge checks)

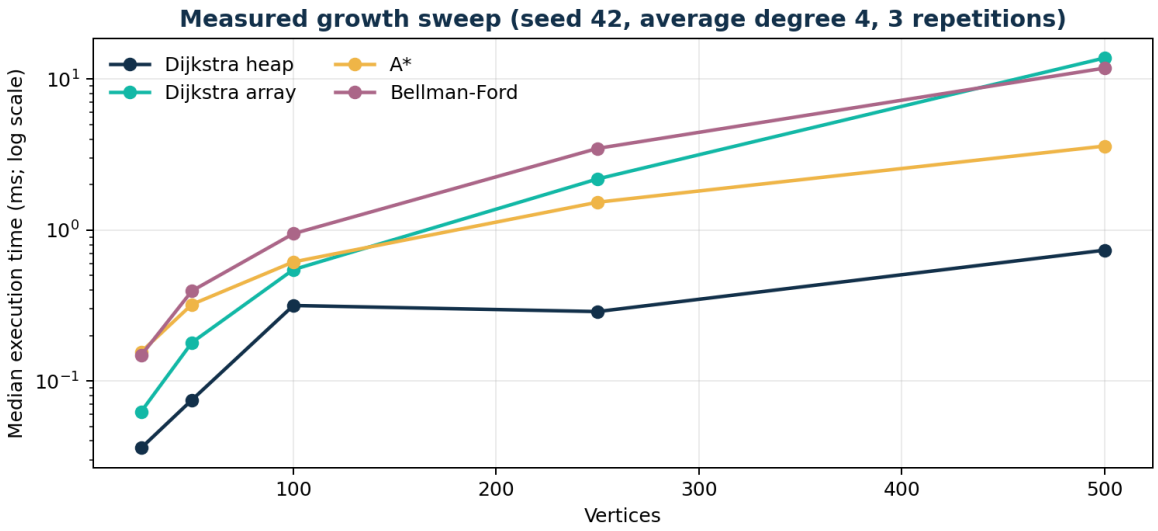


Figure 18. Measured benchmark growth sweep (log scale)

The data show the array implementation's selection overhead becoming more visible at larger sizes, while Bellman-Ford's edge checks grow across repeated relaxations. Heap and array Dijkstra have identical edge-check counts on these cases because the shortest-path instances share the same graph and route target; A* does not necessarily reduce work when its heuristic is weak. Results vary with CPU, Python build, process load, graph generation and seed; the chart is not a cross-hardware performance claim.

4.1.12 Testing and Validation

The complete pytest run executed 16 tests and passed all 16 (0 failures) in 2.07 seconds. The tests cover heap/array Dijkstra path agreement and unreachable/negative cases; A* agreement and lower bounds; BFS/DFS and components; Kruskal forest and DSU; Edmonds-Karp known flow, reverse residual edges and min cut; Bellman-Ford negative edges/cycles; and API/service workflows in the API test module.

Verification level	Observed result	Evidence / note
Algorithm unit tests	Passed	Shortest paths, traversal, DSU/MST, flow, Bellman-Ford
API / integration tests	Passed within suite	Flask test client; input and workflow checks
Full automated suite	16 passed, 0 failed	pytest -q; 2.07 seconds
Manual browser workflows	Representative workflows completed	Dispatch, hospital reservation, evacuation, Dijkstra trace and benchmarks
Connectivity UI form	Failed in manual UI check	Missing source field name; HTTP 400

Table 13. Verification summary

4.1.13 Application Interface Screenshots

The following ten images are authentic captures from the locally running application. They were cropped to the application viewport for legibility; no interface content was fabricated.

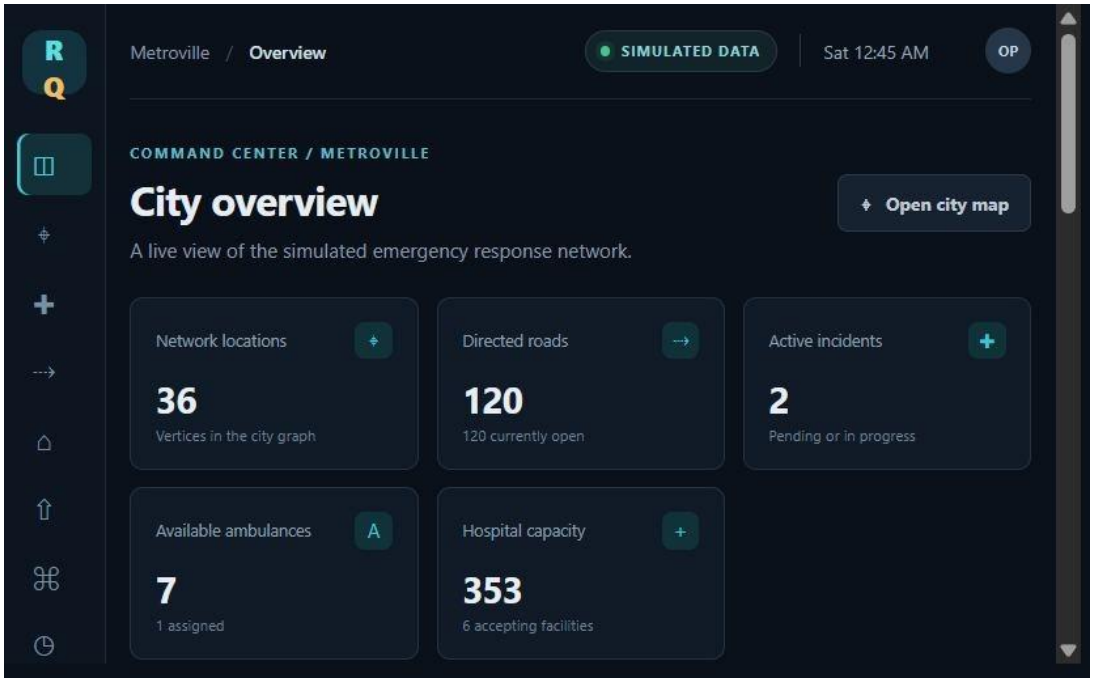


Figure 19. Command overview and simulated network KPIs

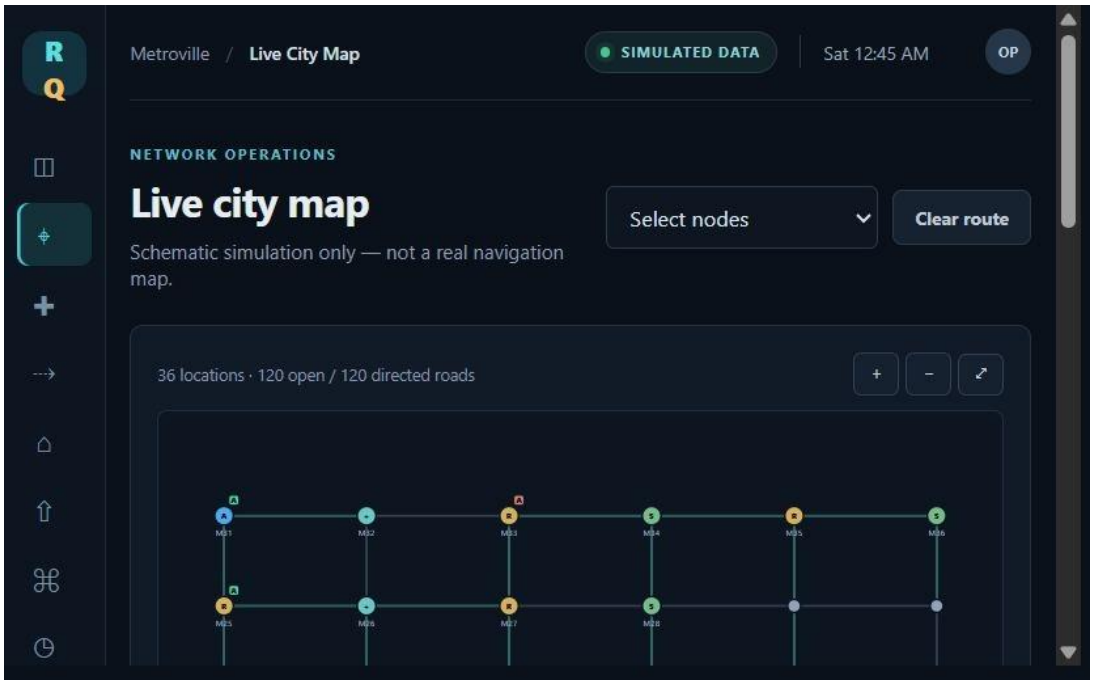


Figure 20. Metroville SVG city network

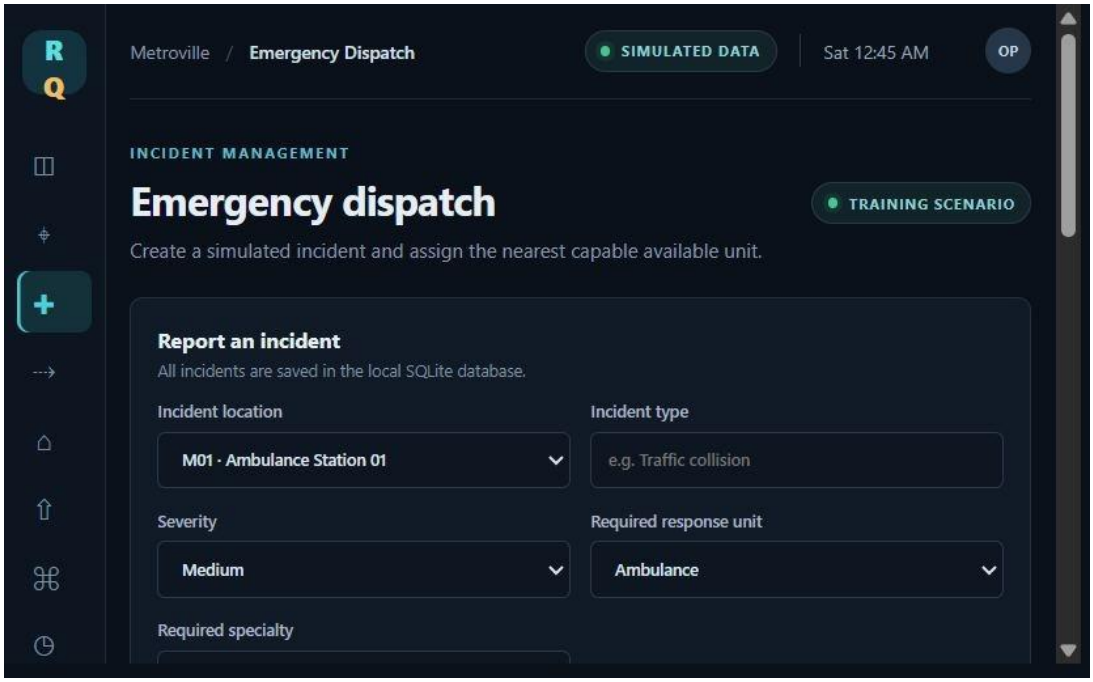


Figure 21. Simulated incident dispatch center

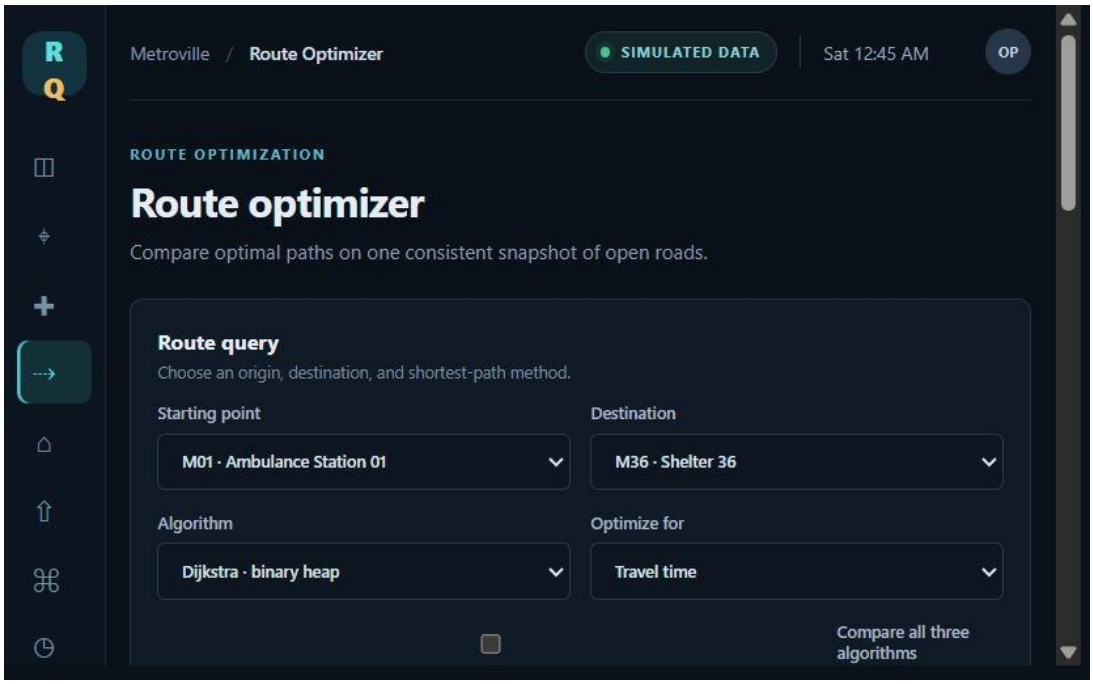


Figure 22. Route optimization panel

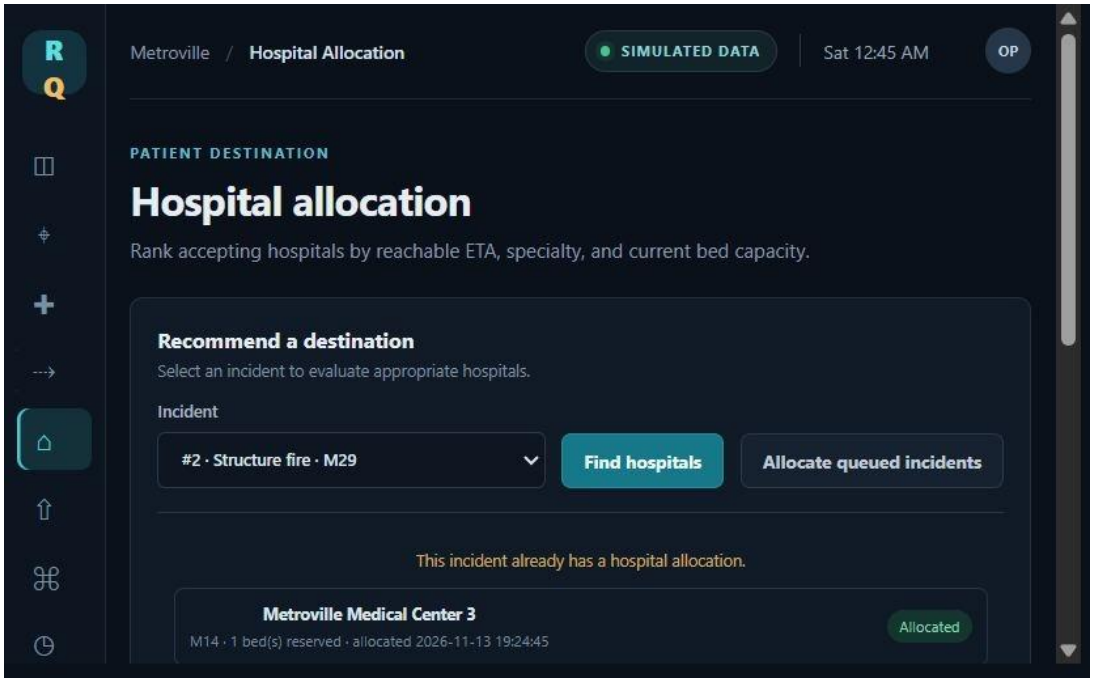


Figure 23. Hospital recommendation and allocation panel

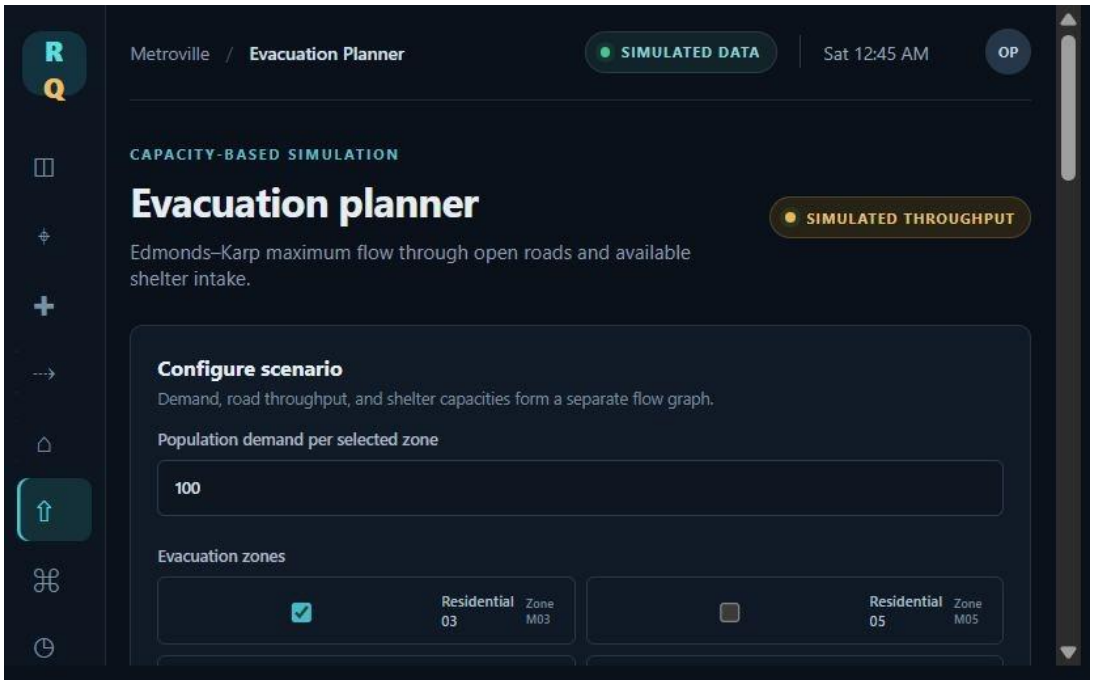


Figure 24. Capacity-based evacuation planner

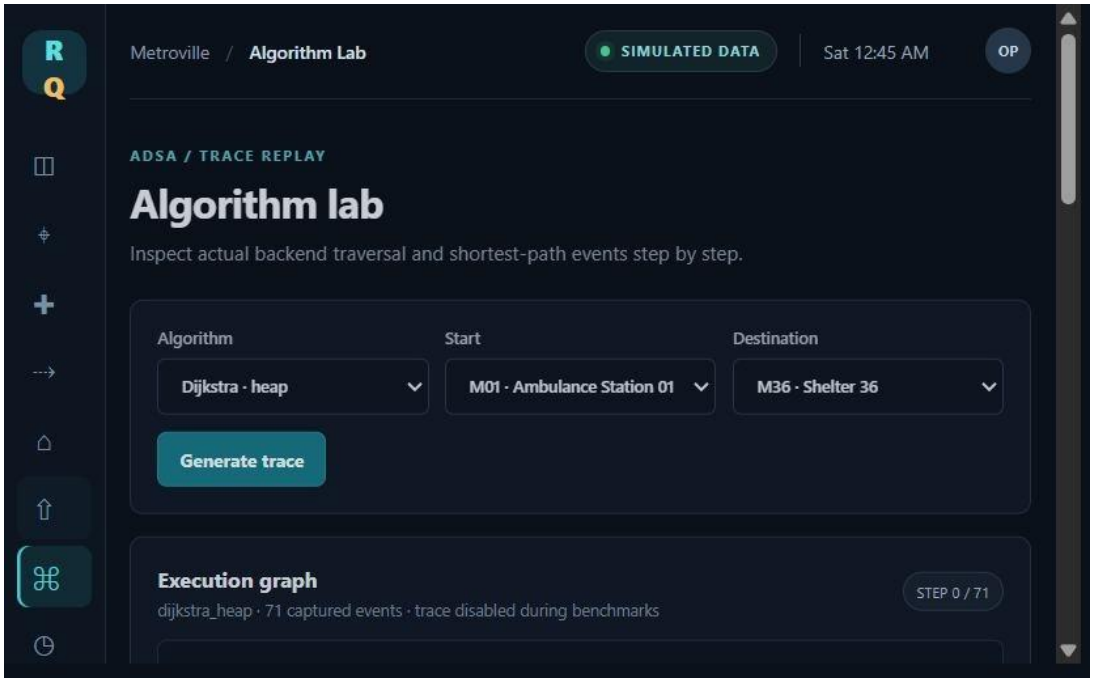


Figure 25. Algorithm trace and visualization lab

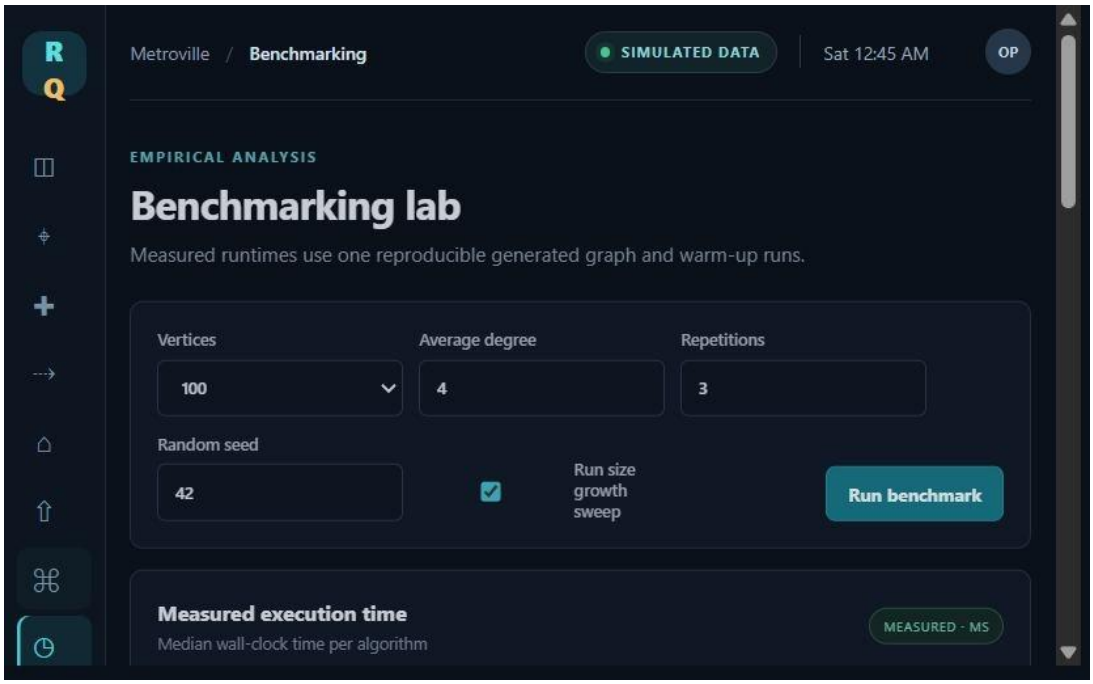


Figure 26. Algorithm benchmarking interface

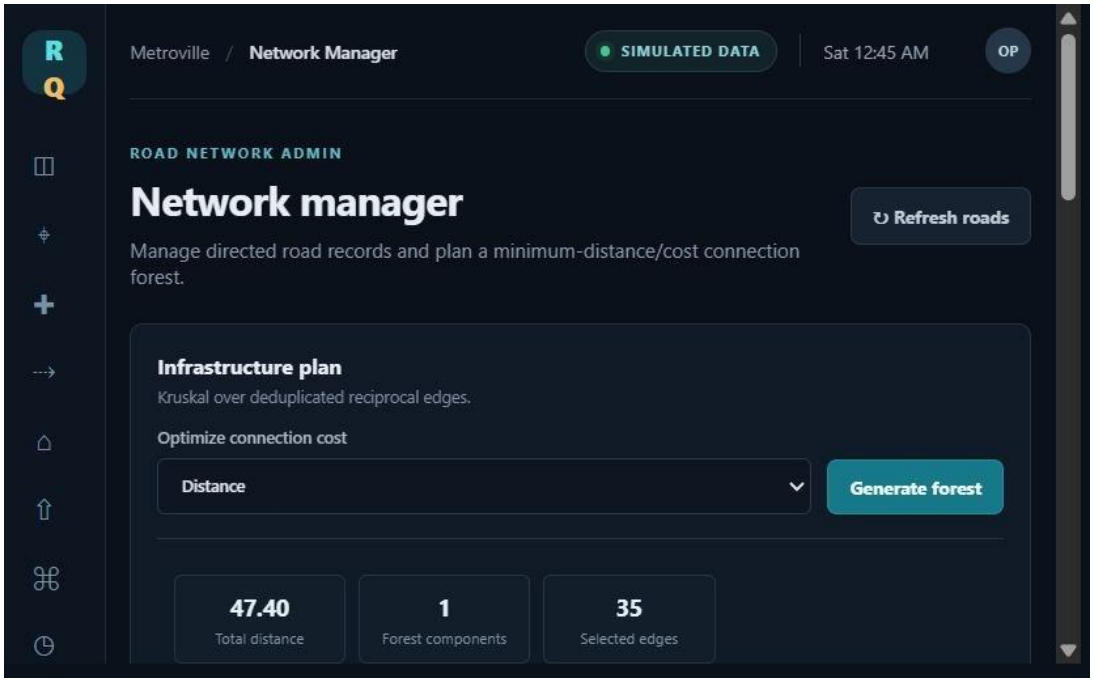


Figure 27. Network management interface

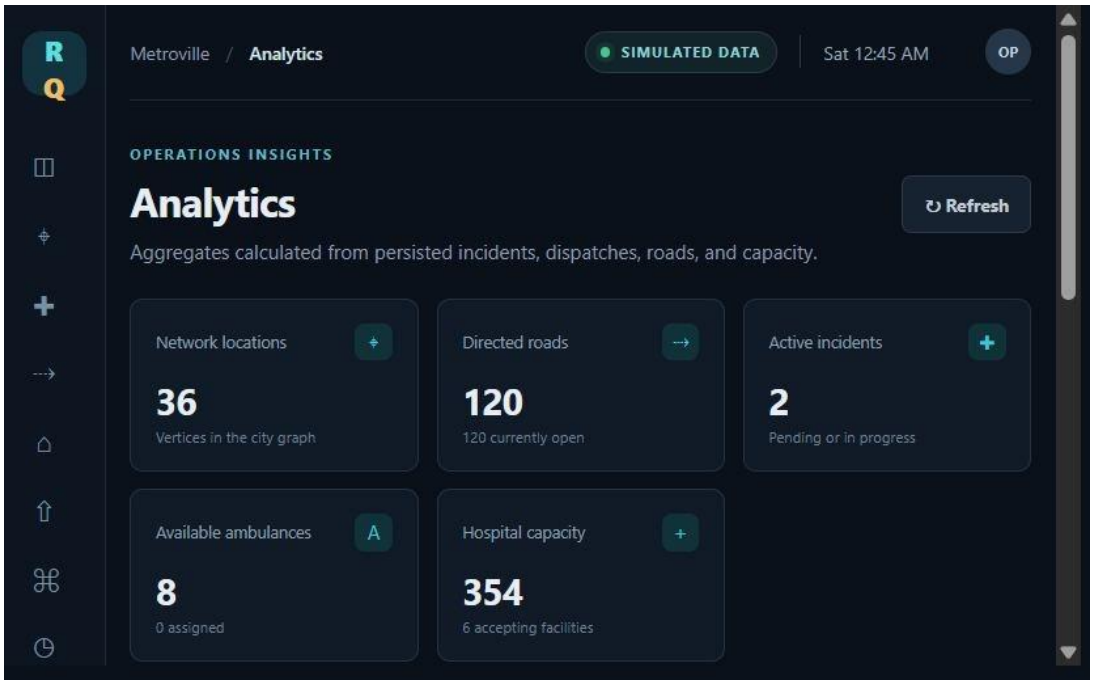


Figure 28. Simulation analytics

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

ResQNet demonstrates how fundamental graph algorithms can be connected to recognizable emergency-response and evacuation-planning workflows in a self-contained academic application. It models a deterministic fictional city as a directed weighted graph, persists simulated resources and events in SQLite, exposes workflows through Flask, and visualizes returned results in a vanilla JavaScript/SVG interface.

Implemented algorithms include two Dijkstra variants, A*, BFS, DFS, Kruskal with DSU, Edmonds-Karp and Bellman-Ford. The service layer uses them for route calculation, ambulance ETA ranking, hospital recommendations, road-closure rerouting, connectivity analysis, infrastructure connectivity planning and capacity-limited evacuation. The local automated suite passed 16 tests. Captured executions demonstrated matching shortest-path costs, a dispatch selection, an eligible hospital recommendation, a 480-unit maximum flow against demand 550, and measured growth trends.

These outcomes meet the academic objectives of implementing, instrumenting and validating multiple algorithmic techniques in an integrated application. They do not establish better real-world emergency outcomes. Expected outcomes were interactive, reproducible demonstrations; actual outcomes include those workflows and a manual connectivity-form defect. All measurements are simulation-specific and hardware-dependent.

5.2 Future Work

Correct the Network Manager connectivity selector field name and add a browser regression test for the submitted source and algorithm fields.

Expand tests for transaction rollback, concurrent dispatch races, hospital capacity races, directed disconnection and invalid flow networks.

Improve benchmark methodology with repeated independent seeds, warm-up disclosure, confidence intervals, larger graph profiles and exportable raw measurements; keep machine/runtime metadata with every run.

Add globally optimized multi-incident hospital assignment (for example, a clearly specified matching/min-cost-flow model), while retaining the current greedy workflow as a baseline.

Explore time-dependent routing and time-expanded evacuation models, uncertainty and risk costs; explicitly validate assumptions before interpreting results.

For any real-world research adaptation, obtain domain-expert review, validated geographic/traffic/capacity data, threat and privacy assessment, accessibility requirements, human-factors studies, and formal safety, security and operational validation. Such work would be a separate project, not a configuration change.

REFERENCES

[1] World Health Organization, Global Status Report on Road Safety 2023, Geneva: WHO, 2023. https://www.who.int/publications/i/item/9789240086517

[2] J. B. Kruskal, “On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem,” Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48–50, 1956. doi:10.1090/S0002-9939-1956-0078686-7.

[3] R. Bellman, “On a Routing Problem,” Quarterly of Applied Mathematics, vol. 16, no. 1, pp. 87–90, 1958. doi:10.1090/qam/102435.

[4] L. R. Ford Jr., Network Flow Theory, RAND Corporation Paper P-923, 1956.

[5] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959. doi:10.1007/BF01386390.

[6] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968. doi:10.1109/TSSC.1968.300136.

[7] J. Edmonds and R. M. Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems,” Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972. doi:10.1145/321694.321699.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[9] ISO/IEC 25010:2023, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model, International Organization for Standardization, 2023. https://www.iso.org/standard/78176.html

[10] ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering, ISO, 2018. https://www.iso.org/standard/72089.html

[11] ISO 22320:2018, Security and resilience — Emergency management — Guidelines for incident management, ISO, 2018. https://www.iso.org/standard/67851.html

[12] ResQNet source code, deterministic database seed, tests and execution evidence, inspected locally for this report, November 2026.

[13] National Disaster Management Authority, Government of India, National Disaster Management Plan, 2019. https://ndma.gov.in/sites/default/files/PDF/ndmp-2019.pdf

[14] Chandigarh University, official website branding asset, https://www.cuchd.in/home-assets/img/logo.png (accessed 14 November 2026).

APPENDIX A — IMPLEMENTATION AND TEST EVIDENCE

The report's primary implementation evidence consists of the algorithm source modules under backend/algorithms/, services under backend/services/, SQLite schema and deterministic seed under backend/database/ and backend/data/, Flask API routes, frontend JavaScript/SVG panels, test modules and the captured execution_evidence.json in docs/report/report_generation/.

Module	Responsibility
backend/algorithms/dijkstra.py	Heap and array shortest-path variants
backend/algorithms/astar.py	A* and Haversine lower-bound heuristic
backend/algorithms/traversal.py	BFS, DFS and undirected connectivity components
backend/algorithms/kruskal.py	Kruskal minimum spanning forest and DSU
backend/algorithms/edmonds_karp.py	Residual matrix, max flow and minimum cut
backend/algorithms/bellman_ford.py	Educational negative-weight shortest paths
backend/services/	Routing, dispatch, hospital, network, evacuation, analytics
backend/database/database.py	SQLite schema and connection lifecycle
backend/data/seed.py	Deterministic Metroville seed and reset
tests/	Algorithm and Flask API tests

Table 14. Key source modules and responsibilities

APPENDIX B — USER MANUAL

B.1 Installation and startup

Install Python 3.10 or later. From the project root, create and activate a virtual environment, install requirements.txt, then run `python run.py`. Open the local URL printed by Flask. The app initializes SQLite and seeds Metroville on first start. `RESQNET_DATABASE` can be set to choose a database location.

B.2 Dashboard and map

Use the overview to inspect simulated counts and recent activity. Open City Map to view locations, facility markers, directed roads and closed-road styling. The schematic map is not geographic navigation.

B.3 Dispatch and incident status

Select or create a simulated incident, request dispatch options, inspect ranked ambulance ETA choices, then confirm dispatch. Use permitted status transitions to progress the incident. Resolve or cancel to release the simulated unit. No real emergency response is contacted.

B.4 Routing and algorithm lab

Choose source, destination, metric (time/distance) and algorithm. Compare Dijkstra heap, Dijkstra array and A*. Inspect route, cost, edge checks and relaxations. Enable trace in the lab to review the returned event sequence. Bellman-Ford examples belong to the educational graph sandbox and do not alter road travel data.

B.5 Hospital recommendation

Request recommendations for a simulated incident. The system filters for acceptance, open-road reachability, free beds and any required specialty. Confirm an eligible facility to reserve a bed. The ranking is algorithmic and not clinical advice.

B.6 Evacuation

Select at least one evacuation zone and one shelter, enter positive integer zone populations and optional non-negative selected-shelter capacity overrides, then run the simulation. Review maximum flow, unserved demand, augmenting paths, capacity use and min cut. Values are hypothetical.

B.7 Network, analytics, reset and safety

The Network Manager edits roads, closes/reopens links, and exposes connectivity and infrastructure analysis. Its connectivity form omits the required source field and returns HTTP 400; valid API calls work. Analytics show simulated history. Reset demo data to restore the seed. Never enter patient data or use output for real emergencies.